

Guia Completo — Desafio 2

Viagens API v2 — PostgreSQL + Spring Security

Desenvolvimento de Sistemas Web · UniSENAI ADS 2026/2

 Este guia cobre: instalação do PostgreSQL, configuração, execução, erros comuns e explicação do código.

PARTE 1 — Instalar e Configurar o PostgreSQL

1.1 Baixar e Instalar

1. Baixe o PostgreSQL

Acesse: <https://www.postgresql.org/download/windows/> e clique em "Download the installer".

Baixe a versão 14 ou superior (64-bit).

2. Execute o instalador

- Clique em Next em todas as telas
- Em "Select Components" deixe marcado: PostgreSQL Server, pgAdmin 4, Command Line Tools
- Em "Password" defina a senha: postgres (ou outra — anote bem)
- Porta: 5432 (padrão — não mude)
- Clique em Next até o fim e aguarde a instalação

3. Verifique a instalação

Abra o PowerShell e rode:

```
psql -U postgres -c "SELECT version();" 
```

Se pedir senha, digite postgres. Se aparecer a versão do PostgreSQL, está instalado!

1.2 Criar o banco de dados

4. Crie o banco viagens_db

No PowerShell:

```
psql -U postgres  
CREATE DATABASE viagens_db;  
\q
```

Ou pelo pgAdmin: clique com botão direito em "Databases" → Create → Database → Nome: viagens_db → Save.

✓ Com o banco criado, o Flyway vai criar as tabelas automaticamente ao rodar a aplicação.

PARTE 2 — Configurar e Abrir o Projeto

2.1 Extrair o projeto

1. Baixe e extraia o ZIP

- Baixe o arquivo viagens-api-v2.zip
- Extraia na Área de Trabalho
- Vai criar a pasta: viagens-api-v2

2.2 Configurar a senha do banco

2. Abra o arquivo application.properties

Caminho: viagens-api-v2\src\main\resources\application.properties

Verifique as linhas:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/viagens_db
spring.datasource.username=postgres
spring.datasource.password=postgres
```

⚠ Se você definiu uma senha diferente durante a instalação do PostgreSQL, troque "postgres" na linha password pela senha que você definiu.

2.3 Abrir no Eclipse

3. Importe o projeto

- Abra o Eclipse
- File → Import → Maven → Existing Maven Projects → Next
- Browse → selecione a pasta viagens-api-v2 → OK
- Marque o /pom.xml que aparecer → Finish
- Aguarde o Eclipse baixar as dependências (pode levar 2-3 minutos na primeira vez)

2.4 Rodar a aplicação

4. Execute o projeto

- No Package Explorer, clique com botão direito no projeto
- Run As → Java Application
- Selecione ViagensApiApplication → OK

Aguarde aparecer no console:

```
Started ViagensApiApplication in X.XXX seconds
```

✓ Se apareceu "Started ViagensApiApplication", a API está rodando em <http://localhost:8080>

PARTE 3 — Testar no Postman

3.1 Configurar autenticação no Postman

Para endpoints protegidos, configure o Basic Auth:

1. Abra uma requisição no Postman

- Clique na aba Authorization
- Em "Auth Type" selecione: Basic Auth
- Username: admin@agencia.com
- Password: admin123

3.2 Testes essenciais

GET público — sem autenticação

```
GET http://localhost:8080/destinos
```

Esperado: 200 OK com lista de 3 destinos em JSON.

POST — apenas ADMIN

```
POST http://localhost:8080/destinos
Authorization: Basic admin@agencia.com / admin123
Content-Type: application/json
{
  "nome": "Lisboa",
  "localizacao": "Lisboa, Portugal",
  "descricao": "Capital histórica de Portugal.",
  "atividades": ["Torre de Belém", "Alfama"]
}
```

Esperado: 201 Created com o JSON do novo destino.

POST sem auth — deve retornar 401

```
POST http://localhost:8080/destinos (sem Authorization)
```

Esperado: 401 Unauthorized — prova que a segurança funciona.

POST com USER — deve retornar 403

```
POST http://localhost:8080/destinos
Authorization: Basic user@agencia.com / user123
```

Esperado: 403 Forbidden — USER não tem permissão para cadastrar.

PATCH avaliação — USER ou ADMIN

```
PATCH http://localhost:8080/destinos/1/avaliacao
Authorization: Basic user@agencia.com / user123
{ "nota": 4.8 }
```

Esperado: 200 OK com a média recalculada.

DELETE — apenas ADMIN

```
DELETE http://localhost:8080/destinos/2
Authorization: Basic admin@agencia.com / admin123
```

Esperado: 204 No Content.

PARTE 4 — Erros Comuns e Soluções

Erro: Connection refused — could not connect to server

Causa: O PostgreSQL não está rodando.

Solução: Abra o Gerenciador de Serviços do Windows (Win+R → services.msc), procure "postgresql" e clique em Iniciar.

Ou no PowerShell como administrador:

```
net start postgresql-x64-14
```

Erro: password authentication failed for user "postgres"

Causa: A senha no application.properties está errada.

Solução: Abra application.properties e corrija a linha:

```
spring.datasource.password=SUA_SENHA_AQUI
```

Erro: database "viagens_db" does not exist

Causa: O banco não foi criado.

Solução: No PowerShell rode:

```
psql -U postgres -c "CREATE DATABASE viagens_db;"
```

Erro: Flyway — Migration checksum mismatch

Causa: Os arquivos SQL de migration foram modificados após já terem sido executados.

Solução: Nunca altere arquivos V1__ ou V2__ após executar a aplicação. Se precisar mudar algo, crie um arquivo V3__.sql com as alterações.

Erro: No compiler provided (JRE instead of JDK)

Causa: O Maven está usando JRE em vez de JDK.

Solução: Use o comando correto para seu ambiente:

```
& "C:\maven\apache-maven-3.9.6\bin\mvn.cmd" "-Djava.home=C:\Program Files (x86)\Eclipse Adoptium\jre-8.0.422.5-hotspot" "package" "-DskipTests"
```

Erro: 401 ao tentar acessar endpoint público

Causa: O Spring Security está bloqueando tudo. Verifique o SecurityConfig.java.

Os endpoints GET /destinos e GET /destinos/{id} devem ter .permitAll() na configuração.

Erro: Cannot find org.postgresql driver

Causa: Dependência do PostgreSQL não foi baixada.

Solução: Clique com botão direito no projeto → Maven → Update Project → Force Update of Snapshots/Releases → OK.

PARTE 5 — Explicação das Partes Importantes do Código

5.1 Entidade JPA — Destino.java

A anotação `@Entity` diz ao Spring que essa classe representa uma tabela no banco de dados. A anotação `@Table(name = "destino")` define o nome da tabela.

```
@Entity
@Table(name = "destino")
public class Destino { ... }
```

O `@Id` marca o campo como chave primária. O `@GeneratedValue` com `IDENTITY` faz o banco gerar o ID automaticamente (equivalente ao `BIGSERIAL` do SQL).

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

O `@ElementCollection` mapeia a lista de atividades para uma tabela separada (atividade), criando um relacionamento 1:N automático sem precisar criar uma entidade Atividade.

```
@ElementCollection(fetch = FetchType.EAGER)
@CollectionTable(name = "atividade", joinColumns = @JoinColumn(name =
"destino_id"))
@Column(name = "nome")
private List<String> atividades;
```

5.2 Repository — DestinoRepository.java

Ao estender `JpaRepository`, ganhamos de graça os métodos: `findAll()`, `findById()`, `save()`, `deleteById()`, `existsById()` — sem escrever uma linha de SQL.

```
public interface DestinoRepository extends JpaRepository<Destino, Long> { }
```

A query customizada com `@Query` usa JPQL (linguagem similar ao SQL mas orientada a objetos). O `LOWER()` e `LIKE` garantem busca parcial e insensível a maiúsculas.

```
@Query("SELECT d FROM Destino d WHERE " +
        "LOWER(d.nome) LIKE LOWER(CONCAT('%', :termo, '%')) OR " +
        "LOWER(d.localizacao) LIKE LOWER(CONCAT('%', :termo, '%'))")
List<Destino> pesquisar(@Param("termo") String termo);
```

5.3 Spring Security — SecurityConfig.java

A classe estende `WebSecurityConfigurerAdapter` para personalizar as regras de segurança. O `@EnableWebSecurity` ativa o Spring Security na aplicação.

O PasswordEncoder com BCrypt é o padrão de mercado para armazenar senhas — nunca armazena a senha em texto puro, sempre como um hash irreversível.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

As regras de autorização definem quem pode acessar o quê. A ordem importa — regras mais específicas devem vir primeiro:

```
.antMatchers(HttpMethod.GET, "/destinos", "/destinos/**").permitAll()
.antMatchers(HttpMethod.PATCH, "/destinos/*/avaliacao").authenticated()
.antMatchers(HttpMethod.POST, "/destinos").hasRole("ADMIN")
.antMatchers(HttpMethod.DELETE, "/destinos/**").hasRole("ADMIN")
```

O httpBasic() ativa a autenticação HTTP Basic — o cliente envia usuário e senha em Base64 no header de cada requisição.

5.4 CustomUserService.java

Essa classe conecta o Spring Security ao banco de dados. Quando alguém tenta fazer login, o Spring chama loadUserByUsername() com o e-mail digitado, que busca o usuário no banco e retorna suas credenciais e permissões.

```
public UserDetails loadUserByUsername(String email) {
    Usuario usuario = usuarioRepository.findByEmail(email)
        .orElseThrow(() -> new UsernameNotFoundException(...));
    return User.builder()
        .username(usuario.getEmail())
        .password(usuario.getSenha())
        .authorities(...) // ROLE_ADMIN ou ROLE_USER
        .build();
}
```

5.5 Flyway — Migrations SQL

O Flyway controla as versões do banco de dados. Ao subir a aplicação, ele verifica quais migrations já foram executadas e roda apenas as novas.

- V1__criar_tabelas_iniciais.sql → cria as tabelas
- V2__dados_iniciais.sql → insere usuários e destinos de exemplo
- Nunca altere um arquivo já executado — crie um V3__, V4__ etc.

As senhas na migration V2 já estão em BCrypt:

- admin123 → \$2a\$10\$N9qo8uLOickgx2ZMRZoMyeljZAgcf17p92ldGxad68LJZdL17lhWy
- user123 → \$2a\$10\$GRLdNijSQMUvI/au9ofL.eDwmoohzzS7.rmNSJZ.0FxO1GkqUKaWm

5.6 Diferença entre v1 e v2 no Service

Na v1, o DestinoService guardava dados em um ConcurrentHashMap na memória — os dados sumiam ao reiniciar.

Na v2, o DestinoService usa o DestinoRepository para persistir no PostgreSQL — os dados ficam salvos permanentemente.

```
// v1 – memória  
private final Map<Long, Destino> repositorio = new ConcurrentHashMap<>();
```

```
// v2 – banco de dados via JPA  
private final DestinoRepository destinoRepository;  
return destinoRepository.save(destino); // persiste no PostgreSQL
```

PARTE 6 — Subir no GitHub e Entregar

1. Crie um repositório no GitHub

- Acesse github.com e logue
- New repository → Nome: viagens-api → Public → Create repository

2. Suba o projeto pelo terminal

```
cd "C:\Users\Usuário\OneDrive\Área de Trabalho\viagens-api-v2"
git init
git add .
git commit -m "Desafio 2 - API com PostgreSQL e Spring Security"
git remote add origin https://github.com/SEU_USUARIO/viagens-api.git
git push -u origin main
```

3. Entregue no AVA

- Acesse o AVA da Unisenai
- Vá na atividade Desafio 2 de DSW
- Cole o link do repositório GitHub
- Prazo: 20/09/2026 às 23h59

 O prazo é HOJE — 20/09/2026 às 23h59. Não deixe para a última hora!